Pedro Moves Cheat Sheet

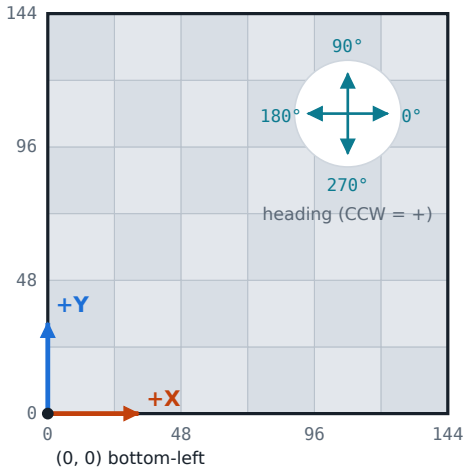
How to move the robot a set distance in autonomous: forward/backward, strafe, rotate, drive-and-turn, and park.

The one idea

Pedro never thinks in "drive 24 inches." Every move is **go from one pose to another pose**. A pose is a spot on the field plus the way the robot faces:

```
Pose p = poseFactory.of(x, y, heading); // in, in, deg
```

To move a set distance, you work out the **end pose** = the start pose plus your distance, then make a path between them. Pedro uses the Pinpoint to check where the robot really is and corrects for slipping, drift and battery level on its own.



The rules of the field

- (0, 0) is the bottom-left corner. +X is right, +Y is up.
- Heading 0° faces +X. 90° faces +Y. Positive turns are **left** (counterclockwise).
- One foam tile = 24 inches. The field is 6 × 6 tiles.
- Each path must **start where the last one ended**.

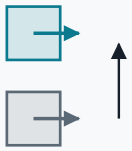
The four kinds of move

Forward / backward



same heading, move along facing

Strafe left / right



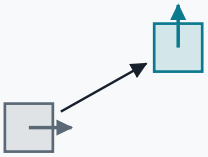
same heading, move sideways

Rotate in place



same spot, new heading

Drive + turn



move and turn together

Move	What changes	Pedro code	Heading option
Forward / backward	X and/or Y along the way the robot faces. Heading stays the same.	`line(a, b).constant(a)`	`.constant(a)` holds a's heading
Strafe left / right	X and/or Y sideways to the robot. Heading stays the same.	`line(a, b).constant(a)`	Same code as forward. Only the end pose differs.
Rotate in place	Only the heading. X and Y stay the same.	`hold(follower, b)` then `waitMs(1000)`	Hold the same spot with a new heading
Drive + turn	Position and heading together.	`line(a, b).linear(a, b)`	`.linear(a, b)` turns smoothly from a to b
Curve	Position along a bend.	`curve(a, ctrl, b).linear(a, b)`	Middle pose(s) bend the path

If you leave the heading option off, Pedro uses **tangent**: the robot turns to face the direction it drives. For mecanum moves you usually want `.constant(...)` or `.linear(...)` instead.

Setting a distance: which numbers change

Find the robot's heading in the left column, then add your distance **d** (inches) as shown. Backward and right are the same rows with a minus sign.

Robot is facing	Forward d	Backward d	Strafe left d	Strafe right d
0° (toward +X)	x + d	x − d	y + d	y − d
90° (toward +Y)	y + d	y − d	x − d	x + d
180° (toward −X)	x − d	x + d	y − d	y + d
270° (toward −Y)	y − d	y + d	x + d	x − d

Any other angle h: forward d → `x + d·cos(h), y + d·sin(h)`. Strafe left d → `x − d·sin(h), y + d·cos(h)`.

Worked examples (robot starts at (24, 24) facing 0°)

```
// 1. Poses: start, then each end point you want
private final Pose startPose   = poseFactory.of(24, 24, 0);
private final Pose fwd24       = poseFactory.of(48, 24, 0);    // forward 24": x + 24
private final Pose left12      = poseFactory.of(48, 36, 0);    // strafe left 12": y + 12
private final Pose turned90    = poseFactory.of(48, 36, 90);   // rotate left 90°, same spot
private final Pose parkPose    = poseFactory.of(48, 60, 90);   // now facing 90°: forward 24" = y + 24

// 2. Paths between them
private Path forward()  { return line(startPose, fwd24).constant(startPose); }
private Path strafe()   { return line(fwd24, left12).constant(fwd24); }
private Path toPark()   { return line(turned90, parkPose).constant(turned90); }

// 3. Run them in order
private Command autoRoutine() {
    return sequential(
            follow(follower, forward()),          // forward 24"
            follow(follower, strafe()),           // strafe left 12"
            hold(follower, turned90),             // rotate in place to 90°
            waitMs(1000),                         //   ...give it time to turn
            follow(follower, toPark())            // drive to park
    );
}
```

Extra imports for the rotate step: `import static com.pedropathing.ivy.pedro.PedroCommands.hold;` and `import static com.pedropathing.ivy.commands.Commands.waitMs;`

Parking

Parking is just the **last path** in `autoRoutine()`. Make a `parkPose` where you want to finish and drive to it from wherever the previous path ended. Your `stop()` already saves the final position to `OpModeStorage` so TeleOp starts in the right place.

Checklist before you run it

- ✓ Each path starts at the pose the previous one ended on. If not, the robot lurches to the start first.
- ✓ Headings in `poseFactory.of(...)` are degrees. Raw numbers in `.constant(...)` or `.linear(...)` are radians, so pass poses instead.
- ✓ For rotate-in-place, tune the `waitMs` time. Too short and the next path starts mid-turn.
- ✓ Test slowly first: add `.with(Constants.foresightConfig.maxPathSpeed.at(0.5))` to a path.
- ✓ Check your poses on the Pedro Field Map before running on the field.